Converge 4A

Automated Appointment Arrangement Application

Project Proposal

Peeranat Thiwongsa 662115034

Supawit Promma 662115049

Bachelor of Science

Software Engineering Program

College of Arts, Media, and Technology

Chiang Mai University

3rd Year

Project Advisor

Apitchaka Singjai, Dr.Techn.

Document History

Version	History	Status	Date	Editable	Reviewer
v0.1.0	- Create Chapter 1,2	C	18-3-2026	PT, SP	-
v0.1.0	- Review Chapter 1,2	R	19-3-2026	-	AS
v0.1.1	- Update Chapter 1	U	22-3-2026	PT, SP	-
v0.1.2	- Update Chapter 2	U	23-3-2026	PT, SP	-
v0.1.3	- Create Chapter 3	C	23-3-2026	PT, SP	-
v0.1.3	- Review Chapter 1,2,3	R	23-3-2026	-	AS
v0.1.4	- Update Chapter 1,2,3	U	24-3-2026	PT,SP	-
v0.1.4	- Review Chapter 1,2,3	R	24-3-2026	-	AS
v1.0.0	- Update Chapter 1,2,3	U	24-3-2026	PT,SP	-
v1.0.1	- Update Chapter 1,2	U	16-4-2026	PT,SP	-
v1.0.2	- Update Chapter 1,2,3	U	17-4-2026	PT,SP	-
v1.0.3	- Update Chapter 1,2,3	U	18-4-2026	PT,SP	-
v1.0.4	- Update Chapter 1,2,3 and add risk management	U	19-4-2026	PT,SP	-

*PT = Peeranat Thiwongsa
*SP = Supawit Promma
*AS = Apitchaka Singjai, Dr.Techn.

Version:
document name vX.Y.Z
X: External published
Y: Test approval
Z: Internal review

Status:
C = Create
R = Reviewed by advisor
U = Update
D = Delete

TABLE OF CONTENTS

Chapter One | Introduction and Rationale **4**
1.1 Motivation 4
1.2 Problem Statement 4
1.3 Purpose 4
1.4 Target Users 5
1.5 Project Scope 5
1.6 Feature Map 6

Chapter Two | Literature Review **8**
2.1 Business Review / Software Review 8
2.2 Architectural Foundations & Prior Art 10
2.2.1 Constraint Satisfaction Modeling for Complex Timetabling 10
2.2.2 Travel Buffer System 10
2.2.3 Heuristic Search and Interactive Explainable AI (XAI) 11
2.3 Tool and Technology Review 11
2.3.1 Frontend Language 11
2.3.2 Frontend Framework 12
2.3.3 Backend Language 12
2.3.4 Backend Framework 13
2.3.5 Database 13
2.3.6 Architectural Pattern 14
2.3.7 Deployment Architecture 14
2.3.8 Containerization 15

Chapter Three | Project Plan **16**
3.1 Objectives 16
3.2 Type of System Users and the Users' Characteristics 16
3.3 Acronyms and Definitions 17
3.4 System Architecture 18
3.5 Use Case Diagram 19
3.6 Product Features 19
3.6.1 Automated Data Ingestion Form 19
3.6.2 Constraint Satisfaction AI Engine 20
3.6.3 Travel Buffer System 20
3.6.4 Branch Capacity Monitoring 21
3.6.5 Converge Hub 21
3.6.6 Automated Disaster Recovery 21
3.7 Software Requirements Specification 22
3.8 SDLC 23
3.9 Limits 24
3.10 Milestones 25
3.11 Risk Management 30

Chapter Four | References **32**

Chapter One | Introduction and Rationale

1.1 Motivation

Multi-branch tutoring centers operate in a highly dynamic environment where instructors frequently rotate between different branches to meet student demand. For these institutions, efficient scheduling is the operational backbone however, as the number of students, teachers, and physical branches grows, the complexity of coordinating weekly classes scales exponentially. Currently, administrators rely on manual systems to manage the schedule.

The booking process begins with a customer selecting a subject, followed by choosing a preferred instructor. The administrator must then manually filter through a static Excel spreadsheet to find and present that specific instructor's available time slots. If none of those times work for the customer, the administrator must repeat the entire filtering process for alternative instructors until a suitable time is found. Once a time is finally selected, the administrator acts as a middleman, sending a minimum of three separate LINE messages: first to the instructor to verify the requested slot, then back to the customer to finalize the booking, and finally to a staff group chat, So the class can be manually logged.

When an instructor is booked for consecutive classes at different branches, administrators must manually calculate and block out their standard 30-minute commute buffer. Managing these transition times by hand across dozens of instructors significantly slows down the booking process. Furthermore, without a branch capacity tracking system, branches are prone to overbooking, leading to situations where teachers and students arrive at a branch only to discover that there is no physical space available.

Ultimately, this repetitive workflow consumes an estimated 1 to 2 hours of administrative time each day and introduces significant risk of human error into the weekly recurring schedule. It forces administrators to manually cross-reference multiple tables within a massive, cluttered spreadsheet just to find a single available teacher for a requested branch.

1.2 Problem Statement

Administrators at multi-branch tutoring schools need an automated, commute-aware scheduling system because the current manual workflow wastes up to 2 hours of administrative time daily. The process requires sending at least three messages per booking, relies on a cluttered spreadsheet that obscures operational insights, and frequently results in branch overbooking. To resolve this, the administration requires an algorithmic system capable of automatically evaluating inter-branch travel times and making sure schedules is optimal.

1.3 Purpose

The primary purpose of the Converge application is to develop an automated scheduling and resource management system that highly mitigates the operational bottlenecks and inefficiencies associated with manual booking in multi-branch tutoring centers.

1.4 Target Users

The target users of Converge are categorized by their level of interaction with the system's core domain, primary users are the main operators who operate the system, while secondary users interact with peripheral interfaces to provide the essential data required for the system to function.

Primary Target Users

1. Administrators are the primary operators who use the system daily to process parent requests. Their goal is to finalize complex, multi-branch schedules instantly while the customer is present. This automated workflow eliminates the operational risks of manual scheduling, specifically preventing branch overbooking and excessive wait times that could harm the school's reputation.

Secondary Target Users

1. Instructors act as data providers for the system. They use a structured web form to submit their weekly availability rather than typing paragraph responses into Google Forms. Allowing the system to enforce commute buffers and generate conflict-free schedules for the administrative staff.

1.5 Project Scope

This project focuses on the design and development of Converge, an enterprise-grade scheduling and resource management system utilizing heuristic optimization for multi-branch tutoring centers. The architecture replaces manual, error-prone workflows with an automated decision-support pipeline:

1. **Automated Data Ingestion Form:** A structured web form that standardizes how instructors submit their availability, replacing unstructured manual entry on Google Form and ensuring clean data for the scheduling engine.
2. **Constraint Satisfaction AI Engine:** A backend scheduling engine that balances variables like instructor availability and branch capacity to generate conflict-free schedules. It includes a transparent decision log, so administrators can easily understand how the engine arrived at its suggestions.
3. **Travel Buffer System:** A configurable module that automatically calculates and applies physical commute times between branches. This ensures instructors are given travel windows without the system needing to rely on third-party mapping APIs.
4. **Branch Capacity Monitoring:** A tracking module that compares concurrent bookings against the physical limits of each branch. It automatically flags full time slots to prevent overbooking and guarantee physical space for every student.
5. **Converge Hub:** A centralized master calendar that replaces static spreadsheets, allowing administrators to view all multi-branch schedules at a glance or instantly filter by specific instructors.

6. **Automated Disaster Recovery:** A scheduled backup system that generates regular database snapshots, ensuring business continuity and rapid restoration in the event of a system failure.

Out of Scope

1. **Self-Service Client Booking:** The system will not include a self-booking system for parents. The client specifically wants administrators to handle scheduling directly, so they can negotiate and sell packages. Furthermore, based on established timetabling research which concludes that complex scheduling requires human intervention to guide the system[1], Converge is designed to be an administrative decision-support tool rather than an app that automatically handles bookings behind the scenes.
2. **Payment Gateway and Financial Processing:** The client already utilizes a proprietary application for financial processing, making a replacement unnecessary. Converge will operate strictly as a decoupled scheduling engine. Furthermore, because their existing application ledger lacks secure APIs, attempting to force an integration between the two systems introduces severe security risk.
3. **Probabilistic Machine Learning / Predictive AI:** The system will not use Large Language Models or predictive AI. These models are prone to data hallucination when processing large datasets. Furthermore, they present severe architectural bottlenecks, self-hosting them might cause processing delays, while using third-party APIs might introduce high token costs. Therefore, the system strictly utilizes deterministic, rule-based AI.
4. **Real-Time Geographic Mapping APIs:** The system will not integrate with third-party map services (e.g., Google Maps) to calculate live travel times. The client explicitly prefers using fixed, standardized travel durations between branches to maintain predictable daily schedules.

1.6 Feature Map

Pain Point	Feature Description	Codename	Success Metric
Teachers submit availability via text paragraphs, admins need to manually interpret and enter data.	An account-less, structured web form where teachers select their identity to submit availability, instantly normalizing the data into the database.	Data-Ingest Form	Drastically reduce manual data entry required for instructor availability.

Continued on next page

(Continued)

Pain Point	Feature Description	Codename	Success Metric
Manual booking workflows waste 1-2 hours daily on complex spreadsheet cross-referencing.	Evaluates variables as a formal CSP to output valid, conflict-free schedules, while exposing Explainable AI (XAI) trace logs directly in the booking view.	Constraint Satisfaction AI Engine	Reduce average booking time to under 2 minutes.
Manual transit calculation is highly vulnerable to human error.	Uses a configurable travel matrix to automatically inject physical commute buffers between branches.	Auto-Buffer	Automate transit enforcement to prevent human errors caused by missed travel buffers.
Lack of capacity tracking leads to overbooking a branch, leaving teachers and students without a physical space upon arrival.	Monitors concurrent bookings against the fixed physical capacity of each branch, flagging full time slots to prevent overbooking.	Capacity-Check	Greatly reduce branch overbooking.
Static spreadsheets create visual complexity and obscure real-time operational insights.	A centralized interface featuring a master calendar view and operational filters for day-to-day oversight.	Converge Hub	Reduce the reliance of Excel to the dashboard for schedule summarization.
Without automated disaster recovery, a system failure or database corruption could permanently erase all multi-branch schedules and halt business operations.	Generates scheduled, asynchronous database snapshots to provide secure, automated recovery points	Auto-Backup	Maintain business continuity by enabling rapid system restoration from automated daily snapshots.

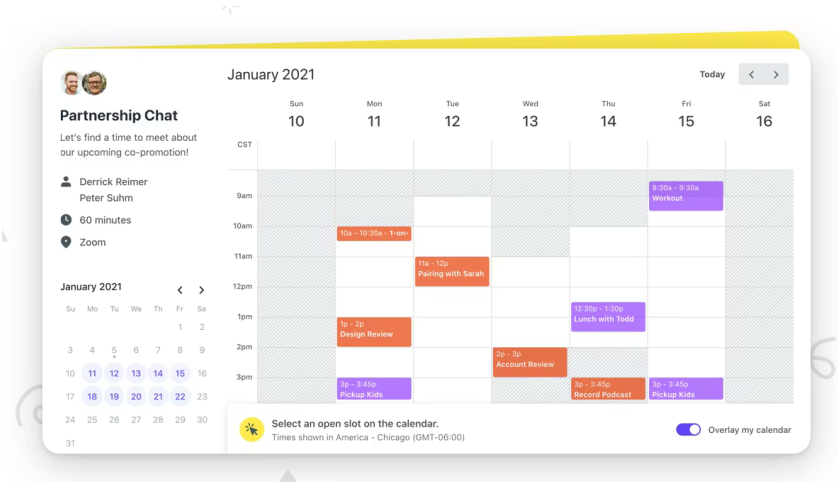
Chapter Two | Literature Review

2.1 Business Review / Software Review

Calendly

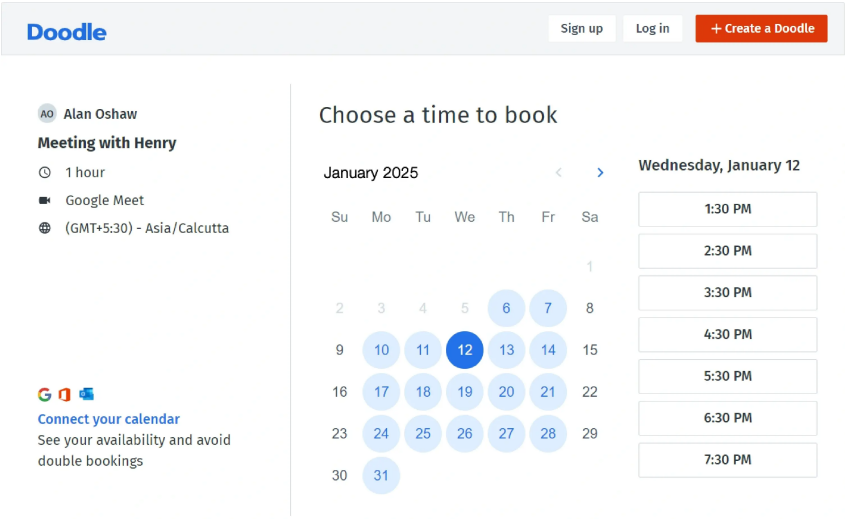
Calendly[2] is an automated scheduling platform that eliminates back-and-forth emails by allowing users to share their availability links, enabling invitees to instantly book open time slots.

Savvycal



Savvycal[3] is a collaborative scheduling tool featuring an interactive calendar overlay, which allows recipients to lay their own calendar over the sender's schedule to easily find mutual availability.

Doodle



Doodle[4] is a group scheduling platform that utilizes a polling system, where an organizer proposes multiple time options and participants vote on their availability to determine the best consensus time for a meeting.

Comparative Table

Feature	Calendly	Savvycal	Doodle	Converge
Automated Data Ingestion Form	No	No	No	Structured Form
Constraint Satisfaction AI Engine	No	No	No	CSP & XAI Logs
Travel Buffer System	Manual	Manual	Manual	Configurable Buffer
Branch Capacity Monitoring	No	No	No	Live Capacity Check
Converge Hub	No	No	No	Multi-Branch View
Automated Disaster Recovery	No	No	No	Asynchronous Snapshots

2.2 Architectural Foundations & Prior Art

Standard scheduling applications only look at calendar availability, they cannot process physical travel times or complex overlapping rules. To solve this, Converge is built upon established Artificial Intelligence and Operations Research methodologies specifically designed for complex logistics.

2.2.1 Constraint Satisfaction Modeling for Complex Timetabling

Timetabling problems are formally modeled as Constraint Satisfaction Problems (CSPs), where a set of variables must be assigned values from defined domains while satisfying a collection of constraints. In the context of course scheduling, variables typically represent assignments such as (instructor, time slot, branch), while constraints enforce rules such as no instructor or room being allocated to more than one class at the same time. This formulation captures the combinatorial nature of scheduling, which has been widely classified as an NP-hard problem, meaning that the computational complexity grows exponentially with the number of variables and constraints[5].

In addition to hard constraints, real-world systems also include soft constraints such as preferred schedules or workload balancing, which are incorporated into an objective function for optimization. Furthermore, educational timetabling problems vary significantly depending on institutional rules, resource limitations, and operational requirements, making each real-world scenario highly unique and difficult to generalize[6].

The Converge system adopts this formal CSP/COP framework by modeling scheduling as a structured decision problem, where instructor availability, branch capacity, and time slots form the variable domains, and constraints enforce feasibility. The system's Constraint Satisfaction AI Engine operates by searching for feasible assignments that satisfy all hard constraints while producing a high-quality schedule under operational conditions.

2.2.2 Travel Buffer System

Classical timetabling formulations primarily focus on logical constraints such as time conflicts and resource allocation; however, real-world scheduling introduces additional constraints that are not always explicitly modeled in standard formulations. Research highlights that practical timetabling systems must incorporate domain-specific constraints to reflect real operational environments[1]. In the case of multi-branch tutoring centers, one critical constraint is the physical movement of instructors between locations.

To address this, the Converge system extends the traditional CSP model by introducing a *Travel Buffer* constraint. This constraint enforces a minimum temporal separation between consecutive assignments of the same instructor across different branches. Formally, if an instructor is assigned to branch A at time t, and branch B requires a travel duration Δ, then any subsequent assignment at branch B must satisfy: $t_2 \geq t + \Delta$

Rather than relying on dynamic external data sources, the system uses a predefined travel time matrix between branches. This design choice ensures deterministic behavior, reduces computational overhead, and maintains consistency with operational policies. By integrating travel

constraints directly into the scheduling model, the system prevents infeasible schedules that would otherwise satisfy only abstract constraints but fail in real-world execution.

2.2.3 Heuristic Search and Interactive Explainable AI (XAI)

Given the NP-hard nature of timetabling problems, exact algorithms are generally infeasible for large-scale instances. As a result, most practical systems rely on heuristic and metaheuristic search strategies to efficiently explore the solution space. Prior research identifies approaches such as greedy heuristics, local search, and constraint-based methods as effective techniques for constructing feasible timetables within reasonable time constraints[1].

A common heuristic principle in timetabling is to prioritize the most constrained variables first, thereby reducing the likelihood of conflicts later in the search process. In the Converge system, heuristic search strategies are used within the Constraint Satisfaction AI Engine to incrementally build feasible schedules while respecting constraints such as availability, capacity, and travel buffers.

In addition, modern timetabling systems recognize the importance of human involvement in the scheduling process, as fully automated solutions may not capture all practical considerations (A Survey of Automated Timetabling). Therefore, Converge incorporates Explainable AI (XAI) principles by providing clear reasoning behind scheduling decisions. Each assignment is accompanied by a traceable explanation of the constraints that influenced it, allowing administrators to understand, validate, and adjust the generated schedules when necessary.

2.3 Tool and Technology Review

2.3.1 Frontend Language

We chose **TypeScript**[7] over JavaScript as the frontend language because its compile-time error checking prevents structural logic failures in our multi-branch scheduling. In making this choice, we accept the tradeoff of a slower initial prototyping phase due to strict type definition requirements.

Aspect / Language	TypeScript	JavaScript	Dart
Typing System	Static Typing	Dynamic Typing	Static Typing
Data Structure	Explicit Contracts	Flexible Objects	Classed Based Model
Development Tooling	Strong IDE support	Basic inference	Strong Tooling
Trade Off	High strictness overhead	Risk of runtime errors	Smaller Web Ecosystem

2.3.2 Frontend Framework

We chose **VueJS**[8] over React because its lower learning curve and built-in tooling allow us to focus on building features rather than configuring the project. In making this choice, we accept the tradeoff of a smaller ecosystem.

Aspect / Framework	VueJS	ReactJS	Angular
Component Structure	Single File Component	JSX	Component + Template
State Management	Proxy-based	Hooks	Services + RxJS
Ecosystem	Small ecosystem (official tools)	Massive ecosystem	Large enterprise ecosystem
Trade Off	Smaller community	Steep learning curve	Heavy framework, verbose

2.3.3 Backend Language

We chose **Golang**[9] over Java and NodeJS as the backend language because its compiled performance and native concurrency model perfectly balance raw execution speed with a low memory footprint. In making this choice, we accept the tradeoff of Go's verbose syntax and manual error handling, which requires writing more boilerplate code than Node.js.

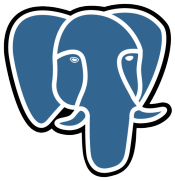
Aspect / Language	Golang	Java	NodeJS
Concurrency Model	Native	Thread-Based	Single-Threaded
Execution Model	Native Binary	JVM Bytecode	JIT Interpreted
Architecture	Minimalist & Explicit	Strict OOP	Highly Dynamic
Trade Off	Verbose syntax	High memory usage	Potential bottlenecks

2.3.4 Backend Framework

We chose the **Gin**[10] framework over Go's native standard library (`net/http`) because its built-in routing and middleware capabilities significantly accelerate the development of our REST API. In making this choice, we accept the tradeoffs of framework lock-in and a slight memory overhead.

Aspect / Framework	Gin	net/http	Fiber
Routing	Advanced	Basic	Express-style, fast
Middleware Management	Built-in	Manual	Built-in
Data Binding & Validation	Automated	Manual	Limited
Ecosystem	Mature	Minimal	Growing
Trade Off	Slight overhead & lock-in	More boilerplate	Less standard, smaller ecosystem

2.3.5 Database



We chose **PostgreSQL**[11] as the database because its native support for range data types is perfectly suited for a scheduling system. In making this choice, we accept the tradeoff of a steeper learning curve and slightly more configuration overhead.

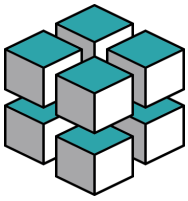
Aspect / Database	PostgreSQL	MySQL	MongoDB
Data Model	Relational	Relational	Document
Querying Complex Data	Advanced	Standard	Aggregation-heavy
Data Type Support	Range Types	Basic Types	Flexible Documents
Trade Off	Steeper learning curve	Limited advanced types	Risk of inconsistency

2.3.6 Architectural Pattern

We chose **Hexagonal Architecture**[12] as the backend architectural pattern because it keeps the core scheduling domain portable and testable in isolation from the web framework. In making this choice, we accept the tradeoff of increased initial complexity, requiring more folders and interfaces upfront.

Aspect / Architect	Modular Monolith	Microservice	SOA
System Complexity	Moderate	High	Moderate–High
Domain Modeling	Well-structured	Distributed	Service-based
Operational Overhead	Low	High	Moderate
Deployment	Single deployable unit	Independently deployable services	Multiple services
Trade Off	Requires discipline	High infrastructure cost	Less flexible than microservices, shared dependencies

2.3.7 Deployment Architecture



We chose a **Modular Monolith**[13] over both Microservices and a standard Monolith to provide a safe evolution path for the system. This approach allows us to properly model the system using Domain-Driven Design (DDD) principles without the immediate operational complexity of a distributed network. In making this choice, we accept the tradeoff of relying on strict developer code discipline to enforce module boundaries, rather than relying on physical infrastructure separation.

Aspect / Architect	Modular Monolith	Microservice	Monolith
System Complexity	Moderate	High	Low
Domain Modeling	Well-structured	Distributed	Unstructured
Operational Overhead	Low	High	Low
Trade Off	Requires discipline	High infrastructure cost	Technical debt risk

2.3.8 Containerization

We chose **Docker**[14] for containerization over traditional bare-metal because it keeps the setup environment consistency across development. By packaging the application and its dependencies into a single image, we eliminate environment-specific bugs. In making this choice, we accept the tradeoff of an increased learning curve, as the entire development team must understand Docker.

Aspect / Strategy	Docker	Podman	Bare Metal
Environment Consistency	High	High	Variable
Resource Isolation	High	High	Low
Deployment Speed	Fast	Fast	Slow
Resource Overhead	Low	Low	None
Security Model	Daemon-based	Daemonless	Host-dependent
Trade Off	Requires daemon, slight overhead	Less mature ecosystem	Environment inconsistency

Chapter Three | Project Plan

3.1 Objectives

The primary objective of this project is to successfully design, develop, and deploy the Converge scheduling system. By the end of the development lifecycle, the project aims to achieve the following measurable goals:

1. **Automate the Booking Workflow:** To drastically reduce the estimated 1 to 2 hours of daily administrative overhead by replacing manual spreadsheet filtering and repetitive messaging with an automated matching engine.
2. **Eliminate Scheduling Conflicts:** To implement a commute-aware algorithm and automated room allocation system that is built to stop double-bookings and enforces cross-branch travel buffers for instructors.
3. **Enhance Operational Visibility:** To deliver a centralized, interactive web dashboard that provides business owners and administrators with a single, real-time source of truth for branch operations.
4. **Implement a Scalable Architecture:** To engineer a robust backend utilizing a Modular Monolith and Hexagonal Architecture, ensuring the core scheduling domain is testable, highly maintainable, and prepared for future organizational scaling.

3.2 Type of System Users and the Users' Characteristics

User Type	Description
Admin	the operators who manage course registrations and schedule bookings based on the incoming requests from parents.
Parent	the users who provide the subject that they want their child to learn to the admin.

User Journey

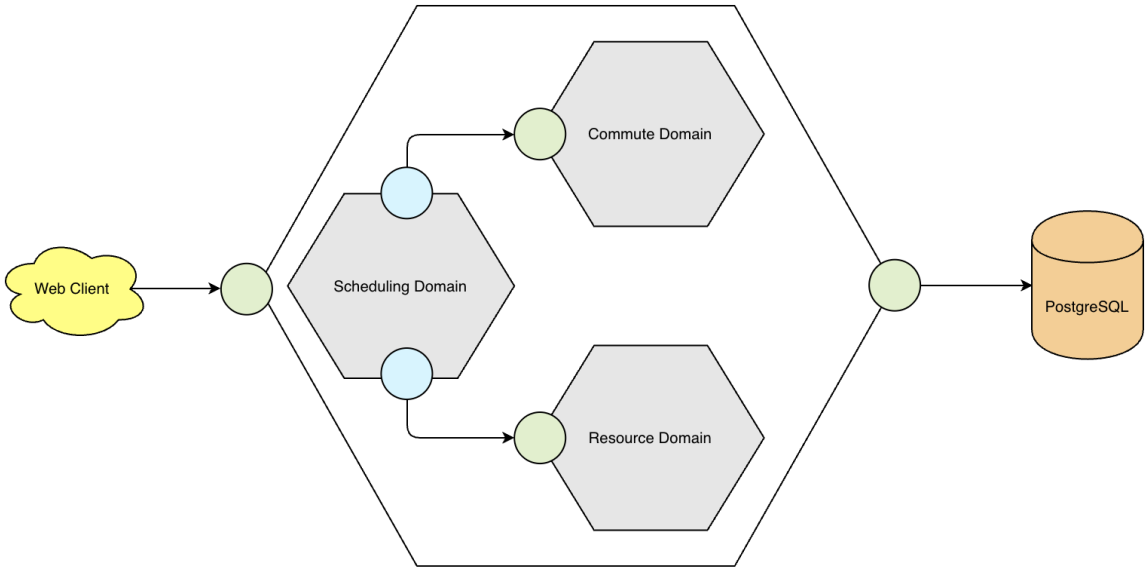
- **As is:** A parent contacts the admin to book a class. The admin has to manually search through a cluttered Excel spreadsheet to find an open time slot. If the time works, the admin then has to manually calculate the teacher's travel time between branches and act as a middleman, sending multiple LINE messages back and forth to confirm with everyone. Even after all this manual work, they might arrive at the branch only to find no physical room is actually available.
- **To be:** A parent contacts the admin to book a class. The admin simply inputs the request into Converge. The system instantly auto-matches the available teacher and student , automatically adds the travel time for the teacher , and permanently locks in a physical room. The booking is confirmed accurately in seconds, completely cutting out the messy spreadsheets and group chats.

3.3 Acronyms and Definitions

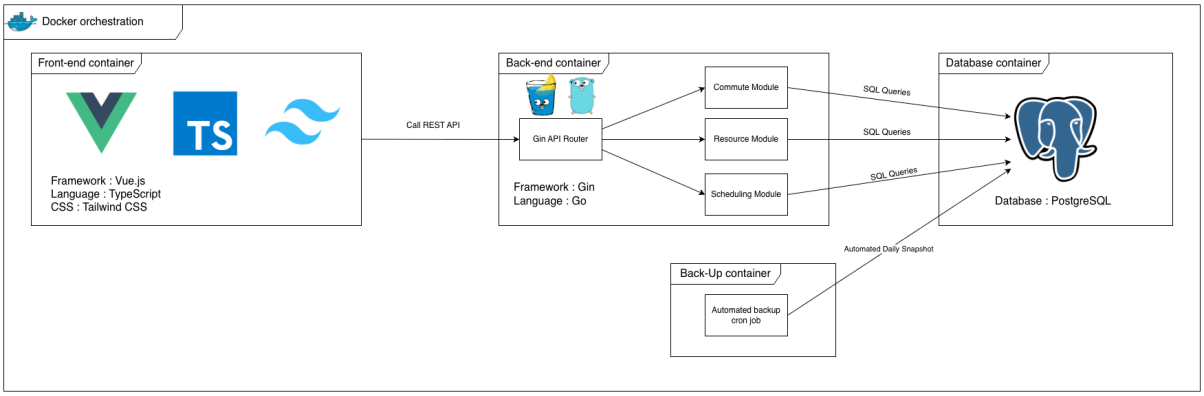
Acronyms	Description
Admin	The operators who manage course registrations and schedule bookings based on the incoming requests from parents.
API	**Application Programming Interface**[15]; used in the context of REST API for communication between the frontend and backend.
Commute Buffer	An automatically calculated travel time inserted between an instructor's consecutive classes at different physical branches.
DDD	**Domain-Driven Design**[16]; a software design approach focused on modeling the system based on the core business domain.
Hexagonal Architecture	A backend architectural pattern that keeps the core scheduling domain portable and testable in isolation from the web framework and database.
Modular Monolith	A deployment architecture where the system is deployed as a single unit but internally structured into well-defined modules.
Parent	The users who provide the subject that they want their child to learn to the admin.
REST	Representational State Transfer[17]; the architectural style used for designing the system's web API.
SaaS	**Software as a Service**[18]; a software distribution model mentioned as out-of-scope for this project.
SDLC	Software Development Life Cycle; the framework used for planning, creating, testing, and deploying the software.
XP	Extreme Programming; an Agile software development methodology focused on software quality, combined with Kanban for this project.

3.4 System Architecture

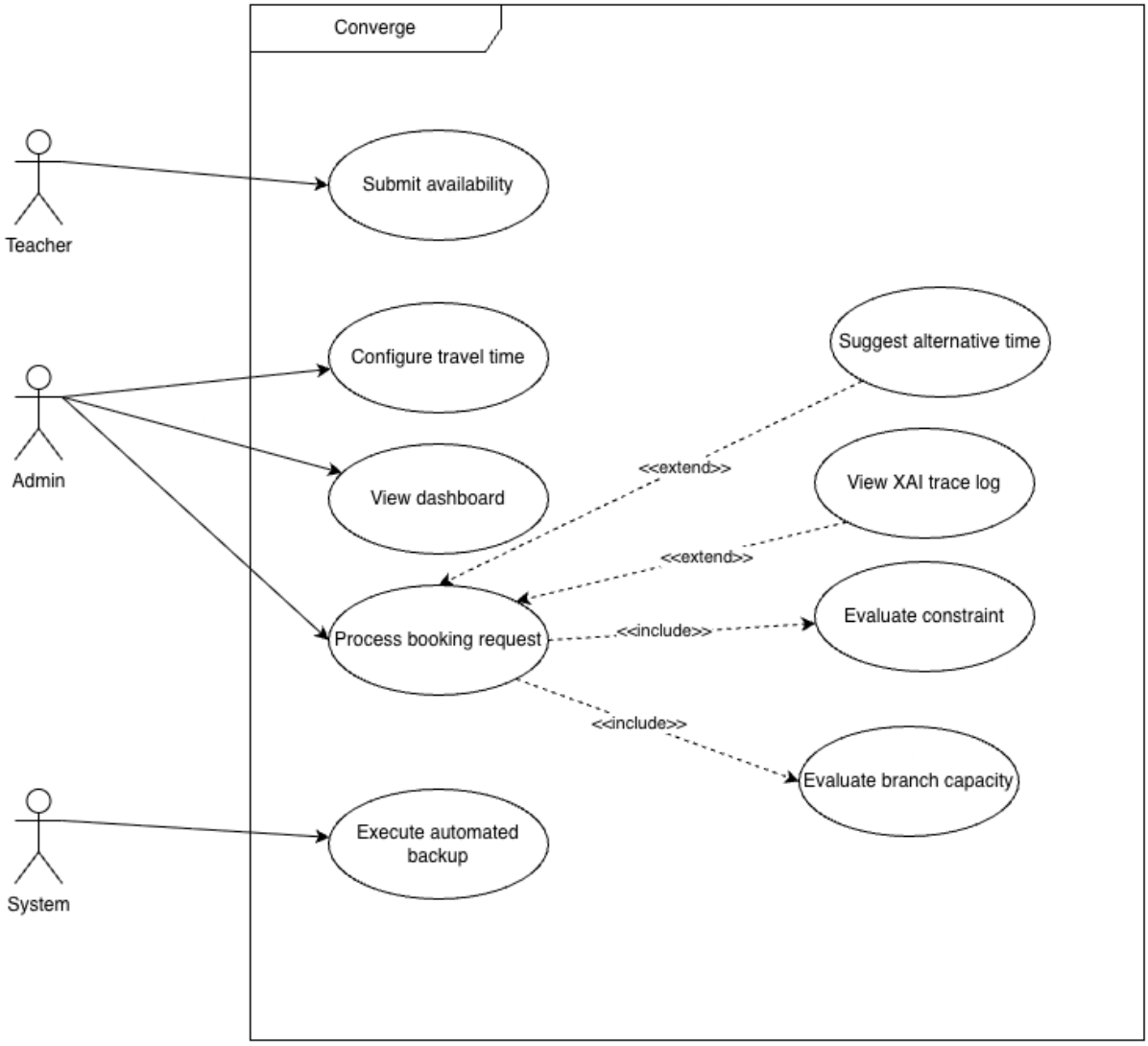
1.Hexagonal Architecture



2.Modular Monolith Deployment Architecture



3.5 Use Case Diagram



3.6 Product Features

3.6.1 Automated Data Ingestion Form

Description: An account-less, structured web form where teachers select their identity to submit their weekly availability.
Actor: Instructors (Teachers)
Details:

- **Structured Input:**
  - Replaces unstructured google form paragraphs with a strict data validation layer.
- **Data Normalization:**
  - Ensures all instructor availability is instantly normalized into discrete binary ma-

trices before entering the database, eliminating the need for administrative manual data entry.

3.6.2 Constraint Satisfaction AI Engine

Description: Evaluates hard limits (instructor availability, branch capacity) to output valid, rule-compliant schedules, rather than attempting to brute-force a single perfect solution.
Actor: Admin
Details:

- **Request Input:**
  - Admin inputs the requested subject and target branch. Specific time and instructor preferences are optional, allowing the system to suggest all possibilities.
- **Constraint-Based Filtering:**
  - Evaluates hard limits (instructor availability, branch capacity) to output optimal schedule rater than attempting to brute-force it.
- **Heuristic Auto-Suggestion:**
  - If exact requests fail, the engine uses heuristic scoring to instantly suggest the next best optimal alternatives in real-time.
- **Explainable AI (XAI) Trace Logs:**
  - Provides a transparent debug view directly within the booking interface, allowing administrators to see the exact CSP constraints that drove the engine's decision.

3.6.3 Travel Buffer System

Description: A configuration-driven module that automatically enforces deterministic travel buffers when an instructor changes branch locations.
Actor: System / Admin
Details:

- **Buffer Configuration:**
  - Provides an interface for administrators to define custom travel durations between specific branch pairs. Any pair without a custom value automatically falls back to a system-wide default.
- **Deterministic Injection:**
  - Automatically shifts an instructor's available start time based on the configured durations, instantly pruning physically impossible transit paths from the search space without relying on external APIs.
- **Visual Buffer Integrity:**
  - The master schedule explicitly displays these calculated commutes as "Travel Needed" to ensure operational clarity for administrators.

3.6.4 Branch Capacity Monitoring

Description: A constraint module that monitors total concurrent classes against the fixed physical capacity of each branch to prevent overbooking.
Actor: System
Details:

- **Capacity Definition:**
  - Administrators define a fixed integer representing the total number of usable spaces at each branch. The system tracks this total threshold rather than assigning specific named rooms.
- **Concurrent Tracking:**
  - Continuously calculates the total number of overlapping bookings at a given branch during a requested time slot.
- **Capacity Feedback & Prevention:**
  - Explicitly flags time slots as "At Capacity" if a new booking would exceed the physical threshold, systematically preventing branch overbooking before a schedule is confirmed.

3.6.5 Converge Hub

Description: A centralized interface that replaces static spreadsheets for day-to-day operations and schedule summarization.
Actor: Admin
Details:

- **Master Schedule View:**
  - A comprehensive, interactive calendar interface designed to phase out legacy spreadsheets. It displays all booked sessions, instructor assignments, and travel buffers across all branches in real-time.
- **Operational Filtering:**
  - Enables filtering by specific instructors, branches, and dates without the visual clutter of standard spreadsheet software.

3.6.6 Automated Disaster Recovery

Description: A backend pipeline that ensures business continuity and enterprise data sovereignty.
Actor: System
Details:

- **Asynchronous Snapshots:**
  - Generates scheduled, automated database snapshots without interrupting real-time scheduling operations.

- **Rapid Restoration:**
  - Provides secure recovery points to quickly restore multi-branch schedules and maintain business operations in the event of hardware failure or database corruption.

3.7 Software Requirements Specification

UC-01 Submit Availability
FR-1.1 The system must display a clearly structured web form for teachers to fill in their availability.
FR-1.2 The system must allow teachers to access and submit free time information without requiring a user account.
FR-1.3 The system requires the instructor to select their identity before beginning to fill in the information.
FR-1.4 The system must validate the data before saving to prevent incorrect data entry.
FR-1.5 The system must automatically convert the idle time data into discrete binary matrices.
FR-1.6 The system needs to record the transformed free time data into the database.

UC-02 Process Booking Request
FR-2.1 The system must allow the administrator to enter the desired subject and target branch for booking.
FR-2.2 The system must evaluate limitations, including instructor availability and branch capacity, to create a compliant schedule.
FR-2.3 The system uses heuristic scoring to immediately suggest the best option if no results matching the specific request are found.
FR-2.4 The system must display the XAI Trace Logs decision records within the booking screen so that the administrator can see the CSP conditions used by the system in making decisions.
FR-2.5 The system needs to calculate the number of concurrent bookings against the maximum capacity set for each branch.
FR-2.6 The system needs to track the total capacity threshold of each branch, instead of specifying individual classroom names.
FR-2.7 The system must display ”At Capacity” during periods when new bookings would cause the number of classes to exceed the branch’s capacity limit.
FR-2.8 The system must systematically prevent branch overbooking before confirming the class schedule.

Business Rules of UC-02
BR-1 An instructor can only be scheduled to teach one class at a time.
BR-2 The total number of concurrent bookings must not exceed the designated capacity limit of that branch.
BR-3 The system will be a decision-support tool and will not perform fully automated booking like a black-box. The final booking confirmation must be processed by an administrator only, allowing for constraints to be negotiated with the customer to confirm.

UC-03 Travel Buffer System

FR-3.1 The system must include an interface allowing administrators to define and customize travel durations between specific branches.
FR-3.2 The system automatically uses a default travel time of 30 minutes for branches that have not been set.
FR-3.3 The system must automatically calculate and adjust the available start time for instructors based on the set travel time.
FR-3.4 The system must automatically exclude physically impossible transit paths from timetable searches without relying on external APIs.
FR-3.5 The system must clearly display the calculated travel time on the master schedule using the words ”Travel Needed”.

Business Rules of UC-03
BR-4 If class scheduling requires an instructor to teach at different branches consecutively, the system must always enforce a commute buffer. If a specific travel duration for that branch pair is not configured, the system will apply a default travel buffer of 30 minutes.

UC-04 View dashboard
FR-4.1 The system must display the Master Schedule View, which is designed to replace traditional spreadsheets.
FR-4.2 The system must display all booked sessions in real-time.
FR-4.3 The system must display instructor assignments on the dashboard.
FR-4.4 The system must display travel buffer information between branches on the dashboard.
FR-4.5 The system must display ”Travel Needed” on the Master Schedule view for the calculated trip.
FR-4.6 The system must have an operational filtering function based on the teacher’s name.
FR-4.7 The system must have a function to filter data by branch.
FR-4.8 The system must have a function to filter data by date.

UC-05 Execute automated backup
FR-5.1 The system must be able to automatically create database snapshots according to a scheduled time.
FR-5.2 The system must perform asynchronous backups to avoid interfering with or interrupting the real-time operation of the scheduling system.
FR-5.3 The system must securely store recovery points.
FR-5.4 The system must be able to support rapid restoration of schedule data for all branches in the event of hardware failure or database corruption.

Non-functional Requirements
1. Security and Authorization
NFR-1 The system requires authentication and only administrator roles can access the system and data to prevent unauthorized access.

3.8 SDLC

Based on our project scope and team size, we decided to use Agile methodologies. It is perfectly suited for a small, remote team like ours that is still in the learning phase and operating

under short deadlines. Agile provides the exact flexibility we need to adapt and succeed.

After confirming Agile as our foundation, we chose to implement a Kanban + XP hybrid style. Kanban tells us what to work on next, while XP tells us how to write it well. Neither alone is enough, but together, they cover everything a 2-person remote team needs to deliver high-quality software.

Furthermore, we realized that a Continuous Flow (Kanban) fits our dynamics much better than fixed 2-week sprints. Since our team members have varying schedules and features naturally finish at different speeds, continuous flow allows us to pull work asynchronously without blocking each other's progress.

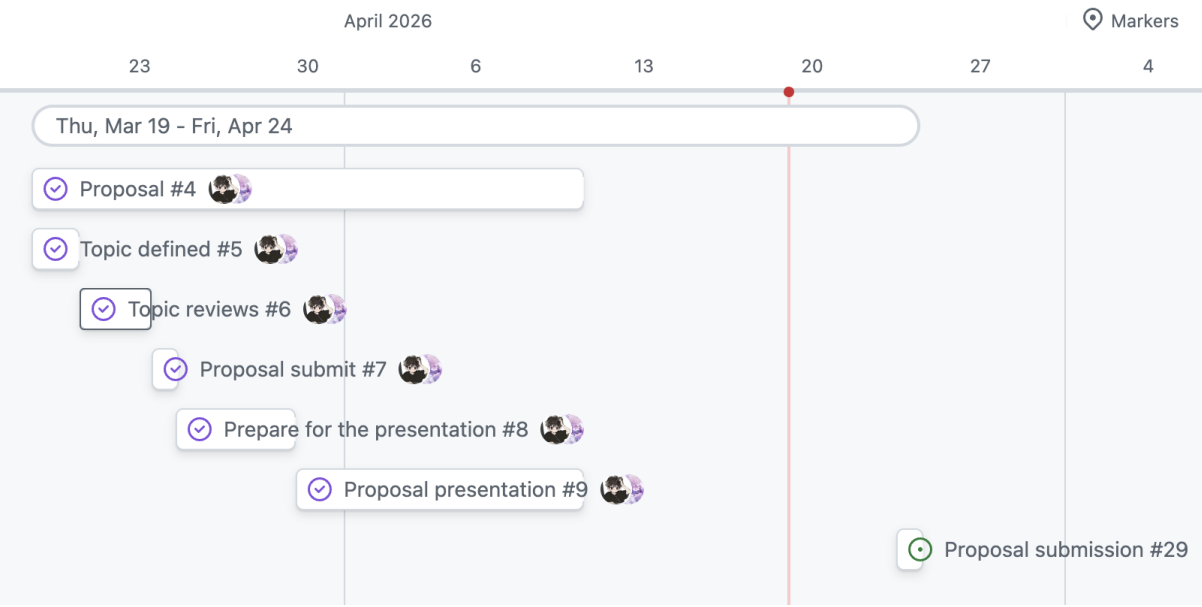
3.9 Limits

1. **Administrative Access Only:** The system is designed as a back-office tool for center managers. It does not include a student-facing portal, tutor login accounts, or self-service booking interfaces for parents.
2. **Non-Financial Scope:** The application does not handle any monetary transactions, including tuition fee collection, payroll processing for tutors, or tax invoicing.
3. **No Automated External Notifications:** While the system calculates the optimal schedule, it does not send automated Email, or LINE messages. All communication with parents and tutors remains a manual process handled by the admin.
4. **Fixed Time Commute Logic:** Commute buffers are calculated based on a configurable branch-pair matrix rather than real-time traffic data, eliminating dependency on external APIs. A default of 30 minutes applies to any pair without a custom value defined.
5. **Single Organization Focus:** The architecture is designed for a single tutoring business with multiple physical branches. it is not a multi-tenant SaaS for multiple independent companies.

3.10 Milestones

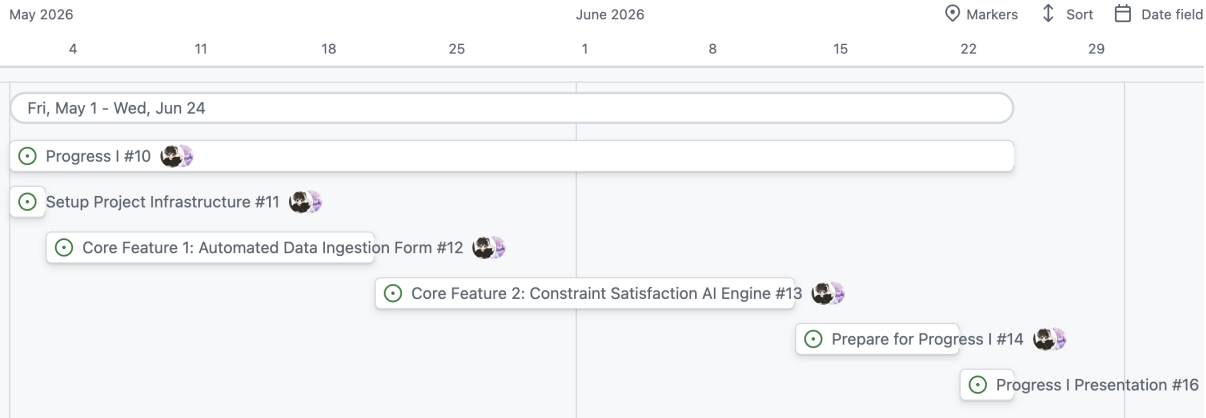
Phase 1: Proposal — (30 March 2026 - 10 April 2026)

Task	Start Date	End Date	Duration	Responsibility
Project Proposal	19 March 2026	10 April 2026	23 days	PT, SP
Topic defined	19 March 2026	10 April 2026	23 days	PT, SP
Topic reviews	21 March 2026	23 March 2026	3 days	PT, SP
Proposal submit	24 March 2026	24 March 2026	1 days	PT, SP
Prepare for the presentation	25 March 2026	29 March 2026	5 days	PT, SP
Proposal presentation	30 March 2026	10 April 2026	12 days	PT, SP
Proposal submission	24 April 2026	24 April 2026	1 days	PT, SP



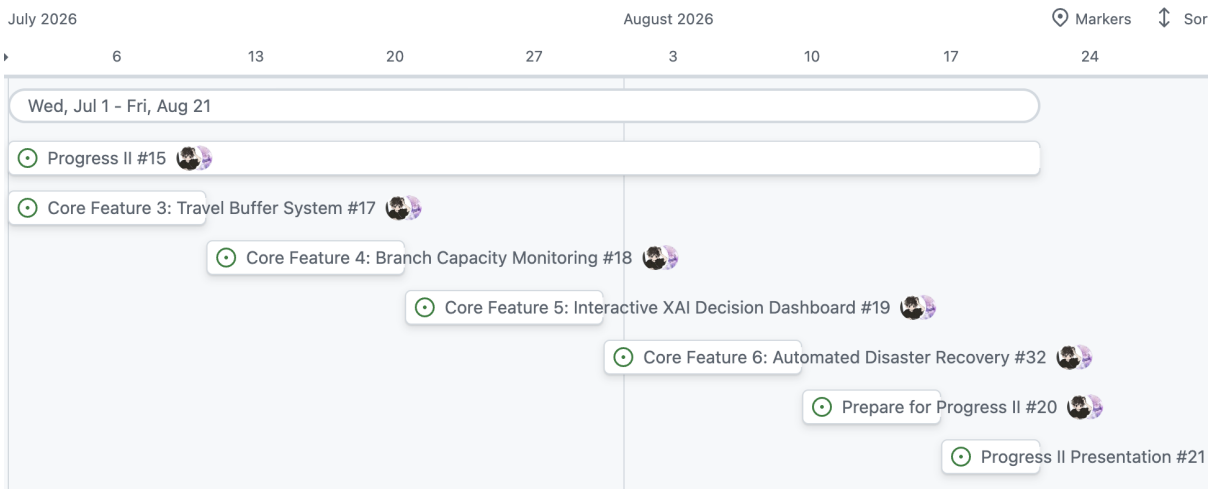
Phase 2: Progress I — (1 May 2026 - 26 June 2026)

Task	Start Date	End Date	Duration	Responsibility
Setup Project Infrastructure	1 May 2026	2 May 2026	2 days	PT, SP
Core Feature 1: Automated Data Ingestion Form	3 May 2026	20 May 2026	18 days	PT, SP
Core Feature 2: Constraint Satisfaction AI Engine	21 May 2026	12 June 2026	22 days	PT, SP
Prepare for Progress I	13 June 2026	21 June 2026	9 days	PT, SP
Progress I	22 June 2026	26 June 2026	4 days	PT, SP



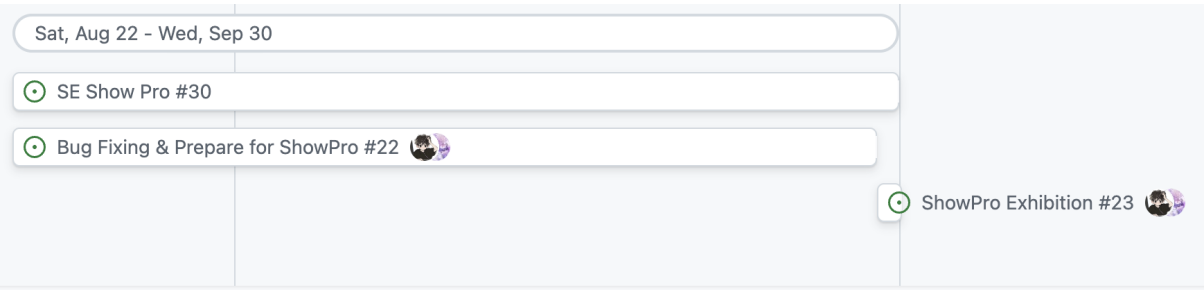
Phase 3: Progress II — (1 July 2026 - 21 August 2026)

Task	Start Date	End Date	Duration	Responsibility
Core Feature 3: Travel Buffer System	1 July 2026	10 July 2026	10 days	PT, SP
Core Feature 4: Branch Capacity Monitoring	11 July 2026	20 July 2026	10 days	PT, SP
Core Feature 5: Interactive XAI Decision Dashboard	21 July 2026	30 July 2026	10 days	PT, SP
Core Feature 6: Automated Disaster Recovery	31 July 2026	9 August 2026	10 days	PT, SP
Prepare for Progress II	10 August 2026	16 August 2026	7 days	PT, SP
Progress II Presentation	17 August 2026	21 August 2026	5 days	PT, SP



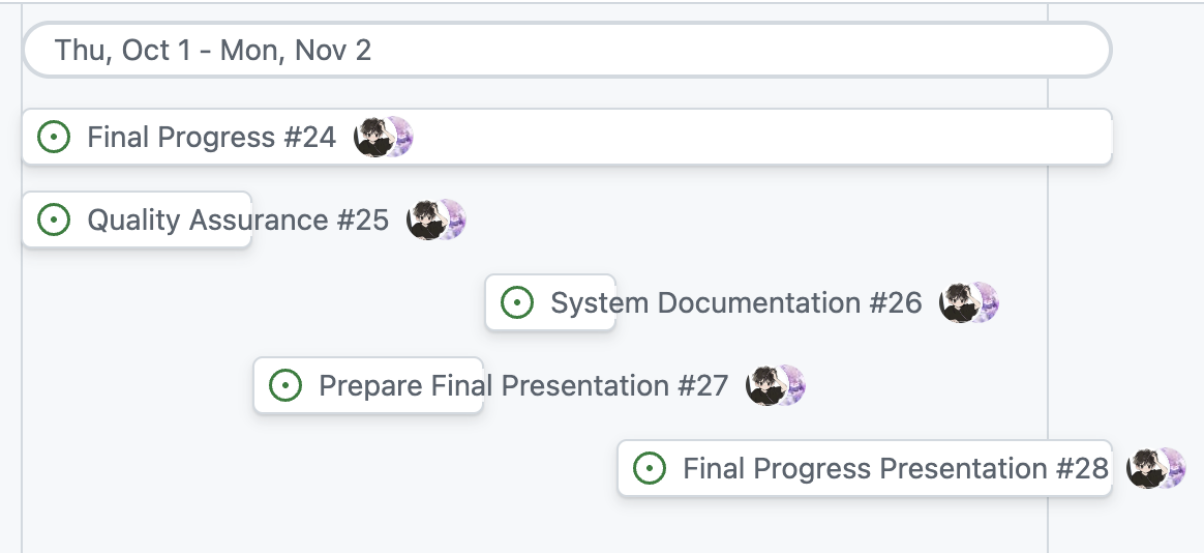
Phase 4: SE Show Pro — (22 August 2026 - 30 September 2026)

Task	Start Date	End Date	Duration	Responsibility
Bug Fixing & Prepare for ShowPro	22 August 2026	29 September 2026	39 days	PT, SP
ShowPro Exhibition	30 September 2026	30 September 2026	1 days	PT, SP



Phase 5: Final Progress — (1 October 2026 - 2 November 2026)

Task	Start Date	End Date	Duration	Responsibility
Quality Assurance	1 October 2026	7 October 2026	7 days	PT, SP
System Documentation	8 October 2026	14 October 2026	7 days	PT, SP
Prepare Final Presentation	15 October 2026	18 October 2026	4 days	PT, SP
Final Progress Presentation	19 October 2026	2 November 2026	15 days	PT, SP



	Pct of task complete (%)	
	Progress I	**Progress II**
Feature 1: Automated Data Ingestion Form	0%	0%
1.1 Structured Input 1.2 Data Normalization		
Feature 2: Constraint Satisfaction AI Engine	0%	0%
2.1 Request Input 2.2 Constraint-Based Filtering 2.3 Heuristic Auto-Suggestion 2.4 Explainable AI (XAI) Trace Logs		
Feature 3: Travel Buffer System	0%	0%
3.1 Buffer Configuration 3.2 Deterministic Injection 3.3 Visual Buffer Integrity		
Feature 4: Branch Capacity Monitoring	0%	0%
4.1 Capacity Definition 4.2 Concurrent Tracking 4.3 Capacity Feedback & Prevention		
Feature 5: Interactive XAI Decision Dashboard	0%	0%
5.1 Master Schedule View 5.2 Operational Filtering		
Feature 6: Automated Disaster Recovery	0%	0%
6.1 Asynchronous Snapshots 6.2 Rapid Restoration		

3.11 Risk Management

Risk	Mitigation Strategy
Team member unavailability due to illness or accident	Kanban workflow allows remaining member to pick up tasks and AI-assisted development as fallback if project completion is at risk

Continued on next page

(Continued)

Risk	Mitigation Strategy
Instructors are used to the current Google Form layout. Converge's new form may confuse them and cause incorrect availability data to be submitted	Design the new form to feel familiar to the existing Google Form. Test with one or two instructors before full rollout. Admin can manually correct any wrong submissions as a fallback
The CSP engine may work fine during development with small test data, but slow down significantly when processing a full week of real availability across all instructors and branches	Set a clear performance target early (e.g. results returned within 3 seconds). Test the engine with realistic data volumes before the Progress I presentation, not just small dummy data

Chapter Four | References

[1] Andrea Schaerf. A survey of automated timetabling. *Artificial Intelligence Review*, 13(2):87–127, 1999.

[2] Calendly official website [online, accessed 18-03-2026] https://calendly.com.

[3] Savvycal official website [online, accessed 18-03-2026] https://savvycal.com.

[4] Doodle official website [online, accessed 18-03-2026] https://doodle.com.

[5] Tarek El-Sakka. University course timetable using constraint satisfaction and optimization. *International Journal of Computing Academic Research (IJCAR)*, 4(3):83–95, 2015.

[6] Sara Ceschia, Luca Di Gaspero, and Andrea Schaerf. Educational timetabling: Problems, benchmarks, and state-of-the-art results. *arXiv preprint arXiv:2201.07525*, 2022.

[7] Typescript official website [online, accessed 18-03-2026] https://typescriptlang.org.

[8] Vue official website [online, accessed 18-03-2026] https://vuejs.org.

[9] Golang official website [online, accessed 18-03-2026] https://go.dev.

[10] Gin official website [online, accessed 18-03-2026] https://gin-gonic.com.

[11] Postgressql official website [online, accessed 18-03-2026] https://www.postgresql.org.

[12] Hexagonal website by aws [online, accessed 18-03-2026] https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/hexagonal-architecture.html.

[13] Modular monolith vs microservice website [online, accessed 18-03-2026] https://www.milanjovanovic.tech/blog/what-is-a-modular-monolith.

[14] Docker official website [online, accessed 18-03-2026] https://www.docker.com.

[15] Api website [online, accessed 18-03-2026] https://aws.amazon.com/th/what-is/api/.

[16] Domain-driven design website [online, accessed 18-03-2026] https://medium.com/devspree/domain-driven-design-934c495e7e0b.

[17] Rest website [online, accessed 18-03-2026] https://developer.mozilla.org/en-US/docs/Glossary/REST.

[18] Saas website [online, accessed 18-03-2026] https://aws.amazon.com/th/what-is/saas/.